

Tools and Programming

CHAPTER 11

11.01

Episode title: **Using Scripting with Pen Testing**

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

SCRIPTING FOR PENETRATION TESTING

- **Why bother with scripts?**

- Automate mundane/repetitive tasks
- Faster
- Less error-prone
- Repeatable

- **What is a script?**

- Interpreted sequence of commands
- Not compiled or assembled
- Easy to code

COMMON SCRIPTING LANGUAGES

- **Bash**

- Bourne Again Shell
 - Command shell for most Linux/MAC OS systems
 - Freely available version of the UNIX Bourne shell (sh)

- **PowerShell**

- Windows-based admin and automation shell
 - Available in Windows since 2006
 - Powerful scripting language

COMMON SCRIPTING LANGUAGES

- **Ruby**

- object-oriented
high-level interpreted general
purpose programming language
 - Influenced by Perl, Smalltalk,
Ada, Lisp

- **Python**

- object-oriented high-level
interpreted general purpose
programming language
 - Extensive available libraries
 - Great intro language

ADDITIONAL RESOURCES

- **Bash**

- Curated list - <https://github.com/awesome-lists/awesome-bash>
- <http://www.commonexploits.com/penetration-testing-scripts/>
- <https://github.com/averagesecurityguy/scripts>
- <https://github.com/bitvijays/Pentest-Scripts>

- **PowerShell**

- <https://www.businessnewsdaily.com/10760-best-free-powershell-training-resources.html>
- <https://blog.netwrix.com/2018/02/21/windows-powershell-scripting-tutorial-for-beginners/>

ADDITIONAL RESOURCES

- **Ruby**

- <https://www.ruby-lang.org/en/>
- <https://hackr.io/tutorials/learn-ruby>
- <http://ruby-for-beginners.rubymonstas.org/index.html>

- **Python**

- <https://learnpythonthehardway.org/>
- <http://shop.oreilly.com/product/9781597499576.do>

SCRIPTING

- **Variables**

- Temporary data storage

- **Substitutions**

- Input parameters and environment variables

- **Common operations**

- Strings and comparisons

- **Logic**

- Looping and flow control

SCRIPTING

- **Basic I/O**

- Read input and write output (file, terminal, and network)

- **Error handling**

- When things don't work

- **Arrays**

- Simple data structure

- **Encoding/decoding**

- Handling special characters

11.02

Episode **Bash Scripting Basics**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

COMMENTS

- **Help you remember what you were thinking**

- All comments start with the '#' character
- Anything after '#' is ignored by the interpreter
- Ex: # This is a comment

VARIABLES

- **varName=value**
 - Ex: name=Michael
- **echo \$name**
- **Common to read data into variables, as opposed to hard-coding too much**
- **Bash variables are untyped**

SUBSTITUTIONS

- **“\$” prefix refers to the contents of an identifier (ex. echo \$name)**
- **Can refer to**
 - Variables `$name`
 - Input parameters `$1`
 - Environment variables `$PATH`
 - Values from utilities `$(whoami)`

SUBSTITUTIONS

- **And, bash will set defaults when no other value is provided**

```
JAVAPATH=${JAVAHOME:=/usr/lib/java}
```

```
OUTPUTDIR=${1:-/tmp}    # IMPORTANT DIFFERENCE
```

COMMON OPERATIONS

- **String operations**

- Concatenate `var="Hello" ; var="$var World"`
- Length `${#string}` OR `expr length $string` ex. `${#name}`
- Extract a substring `echo ${string:position}` ex. `${name:3}`
- Replacing substring `${string/substring/replacement}` ex. `${name/ch/xx}`

- **Compound operations**

- AND: `-a`
- OR: `-o`

COMPARISONS

- `if [ "$varA" -eq "$varB" ]`
- Equal: `-eq` OR `==`
- Not equal: `-ne` OR `!=`
- Greater than, greater than or equal to: `-gt` OR `>`,
`-ge` OR `>=`
- Less than, less than or equal to: `-lt` OR `<`, `-le` OR `<=`
- Not null (empty string): `-n`
- Null (empty string) : `-z`

LOGIC

- **Looping – for**

```
for var in list  
do  
    Statement(s)  
Done
```

- **Examples**

```
for i in 1 2 3 4 5
```

```
for i in $(seq 1 5)
```

FLOW CONTROL

```
if condition
then
    commands

elif condition
then
    commands

else
    commands
fi
```

```
if name=Michael
then <run some command>
```

If name doesn't equal Michael...

```
elif name=Mary
then <run some command>
```

If name doesn't equal Michael OR Mary...

```
else
    then <run some command>
fi
```

BASH if CONDITIONS

Expression	Description
-d file	True if file is a directory
-e file	True if file exists
-f file	True if file exists and is a regular file
-z string	True if string is a null (empty) string
-n string	True if string is not a null (empty string)
stringA = stringB	True if strings are equal
stringA != stringB	True if strings are not equal

BASH SCRIPTING

- **test / []**

- if test -eq \$name "Michael"

- if [\$name = "Michael"]

- **break**

- Exits the current loop iteration

- **exit**

- Exits a script and returns a value (exit code)

BASH PORT SCANNER



```
portscan.sh
File Edit Search Options Help
#!/bin/bash

target=$1
minPort=$2
maxPort=$3

function scanports
{
for ((counter=$minPort; counter<=$maxPort; counter++))
do
    (echo >/dev/tcp/$target/$counter) > /dev/null 2>&1 && echo "$counter open"
done
}

scanports
```

11.03

Episode **Bash Scripting Techniques**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

BASH SCRIPTING I/O

- **I/O – File vs. terminal vs. network**

- Input from a terminal

```
read -p "Enter your name:" name ; echo "Hi, " $name
```

- Input from a file

```
input="filePathName"  
while IFS= read -r f1 f2 f3 f4
```

- Input from the network

```
while read -r inline < /dev/ttyS1
```

ERROR HANDLING

- **Error handling**

- “\$?” is the exit status of a script we just ran

```
if [ “$?” = “0” ] then
```


ARRAYS

```
bashArray = (val1, val2, val3)
```

OR

```
declare -a bashArray = (val, val2, val3)
```

```
for i in 1 2 3
do
    echo ${bashArray[$i]}
done
```

ENCODING/DECODING

- **locale – shows local related environment variables**
- **Can change assignment of LANG for local character encoding**
 - Allows bash to accept special characters (i.e. LANG=da_DK.UTF-8)

ENCODING/DECODING

- Can use openssl or base64 to encode and decode strings (base64)

Encoding:

```
echo string | base64
```

OR

```
base64 <<< string
```

Decoding:

```
echo string | base64 --decode
```

OR

```
base64 -d <<< string
```

BASH: PUTTING IT ALL TOGETHER

- Port scanner in bash



```
portscan.sh
File Edit Search Options Help
#!/bin/bash

target=$1
minPort=$2
maxPort=$3

function scanports
{
for ((counter=$minPort; counter<=$maxPort; counter++))
do
    (echo >/dev/tcp/$target/$counter) > /dev/null 2>&1 && echo "$counter open"
done
}

scanports
```

11.04

Episode **PowerShell Scripting Basics**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

COMMENTS

- **Helps you remember what you were thinking**

- Single-line comments start with the “#” character
- Multi-line comments look like this: <# comment #>

VARIABLES

- **Variables**

- Variable names always start with "\$" ex. \$name = 'Michael'
 - OR \$numberList = 1,3,5,7
- Write-Host \$name \$numberList
- gci variable # lists all defined variables
- Valid data types: [Array], [Bool], [DateTime], [Int], [Int32], [String] (and more)

SUBSTITUTIONS

- **Environment variable – Get-Item**
Env:varName

- Reference with \$Env:varName

- **Input parameters**

```
param (  
    [string]$server = "10.10.10.0",  
    [Parameter(Mandatory=$true)][string]$username,  
    [string]$password = (Read-Host "Input password, please")  
)
```


COMMON OPERATIONS

- **String operations – strings are objects**

- Concatenate `"Hello" + " " + "world"`
- Length `("Hello world").Length`
- Substring `("Hello world").Substring(2,5)`
- Replace substring `("Hello world").Replace("Hello","Greetings")`

COMPARISONS

- **if [“\$varA” -eq “\$varB”]**
- **Equal: -eq**
- **Not equal: -ne**
- **Greater than, greater than or equal to: -gt , -ge**
- **Less than, less than or equal to: -lt , -le**
- **Wildcard match: -like**
- **Match a portion of a string: -match**
- **Logical operators -and -or -not (or !)**

LOGIC

- **Looping – For, While, Do-While, Do-Until**

```
For ($i=0; $i -lt $colors.Length; $i++) { cmds }
```

```
Foreach ($i in $range) { cmds }
```

```
While ($true) { cmds }
```

```
Do { cmds } While ($i -le 10)
```

```
Do { cmds } Until ($i -gt 10)
```

LOGIC

- **Flow control**

```
if (condition) {  
    statements  
} elseif (condition) {  
    statements  
} else {  
    statements  
}
```

I/O

- **File vs. terminal vs. network**

- Input from a terminal

```
$firstName = Read-Host -Prompt 'Enter first name'  
Write-Host $firstName
```

- Input from a file

```
$lines = Get-Content filename  
Out-File -FilePath filename -InputObject $lines -Encoding ASCII
```

- Input from the network

```
$socket = new-object System.Net.Sockets.TcpClient($ip, $port)  
If($socket.Connected) { }
```

ERROR HANDLING

- Try/catch

```
try {  
  Command  
}  
catch {  
  errorHandler commands  
}
```

ARRAYS

```
$PSarray=@(1.3.5.7.9);  
$PSarray.Length  
for ($i = 0; $i -lt $PSarray.Length; $i++) {  
    $PSarray[$i]  
}  
foreach ($element in $PSarray) {  
    $element  
}
```

POWERSHELL SCRIPTING

- **Encoding/decoding**

```
$OutputEncoding = [System.Text.Encoding]::Unicode
```

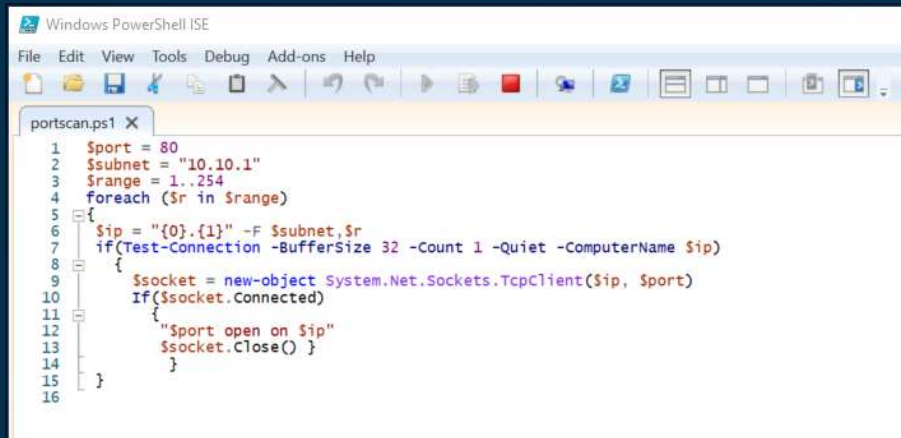
- **Base64 encoding**

```
$Text = 'Hello world'  
$Bytes = [System.Text.Encoding]::Unicode.GetBytes($Text)  
$EncodedText = [Convert]::ToBase64String($Bytes)
```

- **Base64 decoding**

```
$EncodedText = 'encodedString'  
$DecodedText =  
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String($EncodedText))
```


PowerShell: Putting it all together



```
1 $port = 80
2 $subnet = "10.10.1"
3 $range = 1..254
4 foreach ($r in $range)
5 {
6     $ip = "{0}.{1}" -F $subnet,$r
7     if (Test-Connection -BufferSize 32 -Count 1 -Quiet -ComputerName $ip)
8     {
9         $socket = new-object System.Net.Sockets.TcpClient($ip, $port)
10        If ($socket.Connected)
11        {
12            "$port open on $ip"
13            $socket.Close()
14        }
15    }
16 }
```

11.05

Episode **Ruby Scripts**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

HOW TO RUN RUBY SCRIPTING

- **Download and install Ruby**
 - <https://www.ruby-lang.org/en/downloads/>
 - Launch Ruby: irb (Interactive Ruby) (ctrl-D to exit)
 - Or, just run Ruby from a web browser - <https://ruby.github.io/TryRuby/>
- **Comments**
 - '#' for single line comments, =begin comments =end (multi-line comments)
- **Variables**
 - name = "Michael"
 - number = 22
 - puts name, number
 - Valid data types: number, string, Boolean, symbol, array, hash

SUBSTITUTIONS

- **Environment variables** puts ENV['PATH']
- **Input parameters** ARGV ARG[0] ARG[1]

```
ARGV.each do |a|  
  puts "Argument: #{a}"  
end
```

- Ruby also has an OptionParser library
- **Values from other utilities** `echo \$PATH`

COMMON OPERATIONS

- **String operations**

- Concatenation `"snow" + "ball"`
- Repetition `"hi" * 3`
- Length `"hello".length`
- Substring (extract or replace) `"hello"[1..3]`

COMMON OPERATIONS

- **Comparisons**

- Equal `==`
- Not equal `!=`
- Greater than, greater than or equal to `>`, `>=`
- Less than, less than or equal to `<`, `<=`

- **Logical operations**

- and `&&`
- or `||`
- not `!`

LOGIC

- **Looping – while, until, for**

```
while condition do  
  statements  
end
```

```
until condition do  
  statements  
end
```

```
for var in expression do  
  statements  
end
```

LOGIC

- **Flow control**

```
if condition then
  statements
elsif
  statements
else
  statements
end
```


LOGIC

```
Case input
  when "A"
    statement
  when "B"
    statement
  else
    statement
end
```

I/O

- **File vs. terminal vs. network**

- Input from terminal `name = gets`

- Input from a file

```
inFile = File.new("filename","r")
inFile.each_line {|line| puts "#{line.dump}" }
inFile.close
```

- Output to a file

```
$stdout << 76 << " trombones" << "\n"
```

- Network I/O

```
client = TCPSocket.open('hostname', 'port')
client.send("string",0)
```

ERROR HANDLING

- **begin / end / rescue**

```
begin
  statements
rescue
  statements if error occurred
else
  statements if no error
end
```

ARRAYS

```
rubyArray = [ "val1", "val2", "val3" ]  
print rubyArray[1]  
print rubyArray.index("val2")  
print rubyArray.last OR print rubyArray[-1]
```

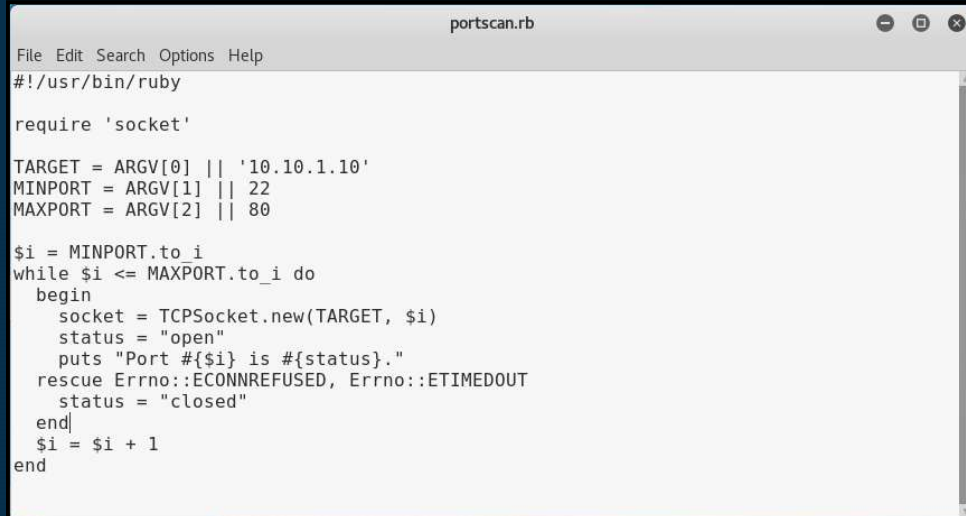
ENCODING/DECODING

Require "base64"

```
encString = Base64.encode64("Hello world!")
```

```
plaintext = Base64.decode(enc)
```

RUBY: PUTTING IT ALL TOGETHER

A screenshot of a text editor window titled 'portscan.rb'. The window has a menu bar with 'File', 'Edit', 'Search', 'Options', and 'Help'. The code is written in Ruby and performs a port scan on a target IP address. It uses ARGV to get command-line arguments for the target IP, minimum port, and maximum port. It then iterates through the ports, attempting to open a TCP socket and reporting the status as 'open' or 'closed' based on the result.

```
File Edit Search Options Help
#!/usr/bin/ruby

require 'socket'

TARGET = ARGV[0] || '10.10.1.10'
MINPORT = ARGV[1] || 22
MAXPORT = ARGV[2] || 80

$i = MINPORT.to_i
while $i <= MAXPORT.to_i do
  begin
    socket = TCPSocket.new(TARGET, $i)
    status = "open"
    puts "Port #{$i} is #{status}."
  rescue Errno::ECONNREFUSED, Errno::ETIMEDOUT
    status = "closed"
  end
  $i = $i + 1
end
```

11.06

Episode **Python Scripts**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

PYTHON SCRIPTING

- **Download and install Python**
 - <https://wiki.python.org/moin/BeginnersGuide/Download>
 - Two versions in use: 2 and 3
 - Launch Python: python (ctrl-D to exit)
- **Comments - all comments start with “#”**
- **Variables**
 - name = “Michael”
 - number = 22
 - print(name + “ “ + str(number))
 - Valid datatypes: numbers, string, list, tuple, dictionary

SUBSTITUTIONS

- **Input arguments (parameters)**

```
import sys
print ("Name of script:", sys.argv[0])
print ("Number of arguments: ", len(sys.argv))
print ("Arguments: ", str(sys.argv))
```

- **Environment variables**

```
import os
extPath = os.environ['PATH']
```

COMMON OPERATIONS

- **String operations**

- Concatenate `string1 + string 2`
- Length `len(string)`
- Extract substring `string[start:end+1]`
- Replace a substring `string.replace(old, new, count)`

COMMON OPERATIONS

- **Comparisons**

- Equal ==
- Not equal != OR <>
- Greater than, greater than or equal to >, >=
- Less than, less than or equal to <, <=

- **Logical operations**

- and
- or
- not

LOGIC

- **Looping – for, while**

```
for i in range(1, 10):  
    print(i)
```

```
while x < 10:  
    print (x)  
    x += 1
```

LOGIC

- **Flow control – if**

```
If var == value:  
    statements  
elif var > value:  
    statements  
else:  
    statements
```

- Notice indentation

I/O

- **File vs. terminal vs. network**

- Input from a terminal

```
name = raw_input('Please enter your name') # map to simple datatype
toppings = input('Which toppings do you want on your pizza?') # maps to complex datatype
```

- Input() will store data in the "best" datatype (i.e. list, etc.)

I/O

- **File vs. terminal vs. network**

- Input from a file

```
f = open('inFile.txt','r')
for line in f:
    do something here
f.close()
```

I/O

- **File vs. terminal vs. network**

- Output to a file

```
f = open('outFile.txt','w')
for i in range(1,11):
    print >> f, i
f.close()
```


I/O

- **File vs. terminal vs. network**

- Input from a network

```
sock = socket.socket(socket.AF_INET,  
socket.SOCK_STREAM)  
If sock.connect_ex((remoteServerIP,  
port)) == 0:  
    print ('Port {}: is Open'.format(port))
```

ERROR HANDLING

- Try / except / finally blocks

```
try:  
    statements  
    raise customErrorObject  
except errorObject:  
    statements  
except customErrorObject:  
    statements  
finally:  
    statements to clean up
```

ARRAYS

```
pythonArray = [10, 20, 30, 40, 50]
Print(pythonArray[-1])    # -1 is last element index
len(pythonArray)
pythonArray.append(60)    # add 60 to the array
pythonArray.remove(30)    # remove element 30
pythonArray.pop(3)        # remove the 4th current element
```

ENCODING/DECODING

```
import base64  
encString = base64.encodestring('Hello world!')  
plaintext = base64.decodestring(encString)
```

Python: Putting it all together



The image shows a screenshot of a text editor window titled "portscan.py". The window has a menu bar with "File", "Edit", "Search", "Options", and "Help". The code is written in Python and is as follows:

```
1 import sys, socket
2
3 target = sys.argv[1]
4 minport = int(sys.argv[2])
5 maxport = int(sys.argv[3])
6
7 def porttry(cur_target, port):
8     try:
9         s.connect((cur_target, port))
10        return True
11    except:
12        return None
13
14
15 for i in range(minport, maxport+1):
16     s = socket.socket(2, 1) #socket.AF_INET, socket.SOCK_STREAM
17     value = porttry(target, i)
18     if value != None:
19         print("Port opened on %d" % i)
20
```

11.07

Episode title: **Scripting Languages Comparison**

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Comments	#	# or <# #>	# or =begin =end	#
Variables – assign	varName=value	\$varName=value	varName=value	varName=value
Variables – display	echo \$varName	Write-Host \$varName	puts varName	print(varName)
Substitution – environment variables	\$envVarName	Get-item Env:varName	ENV['varName']	Os.environ['varName']

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
String length	<code>\${#string}</code>	<code>(string).Length</code>	<code>string.length</code>	<code>len(string)</code>
String – substring	<code>\${string:position}</code>	<code>(string).Substring(start,end)</code>	<code>string[1..3]</code>	<code>string[start:end+1]</code>
String – replace substring	<code>\${string/substring/replacement}</code>	<code>(string).Replace(substr,replStr)</code>	<code>string[1..3] = replStr</code>	<code>string.replace(old, new, count)</code>
AND/OR	<code>-a / -o</code>	<code>-and, -or, -not !</code>	<code>and &&, or , not !</code>	<code>and, or, not</code>
Comparisons	<code>-eq (==), -ne (!=), -lt (<), -le (<=), -gt (>), -ge (>=)</code>	<code>-eq, -ne, -gt, -ge, -lt, -le</code>	<code>==, !=, >, >=, <, <=</code>	<code>==, != (<>), >, >=, <, <=</code>

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Looping	For	For, While, Do-While, Do-Until	while, until, for	for, while
Flow control	if condition then commands elif commands else commands fi	if (condition) { statements } elseif (condition) { statements } else { statements }	If condition then statements elsif statements else statements end	if condition: statements elif condition: statements else: statements

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Input – file	Input="filename" While IFS=read -r f1 f2 f3	\$lines = Get-Content filename Out-File -FilePath filename -InputObject \$lines -Encoding ASCII	inFile = File.new("filename","r") inFile.each_line { line puts "#{line.dump}" } inFile.close	f = open('inFile.txt','r') for line in f: do something here f.close()
Input – terminal	Read -p "Prompt:" var	\$firstName = Read- Host -Prompt 'Enter first name'	name = gets	name = raw_input('Please enter your name')

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Input – network	<pre>While read -r inline < /dev/ttyS1</pre>	<pre>\$socket = new-object System.Net.Sockets.Tc pClient(\$ip, \$port) if(\$socket.Connected) { }</pre>	<pre>client = TCPSocket.open('hostn ame', 'port') Client.send("string",0)</pre>	<pre>sock = socket.socket(socket.AF_ INET, socket.SOCK_STREAM) If sock.connect_ex((remote ServerIP, port)) == 0: print ('Port {}: is Open'.format(port))</pre>

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Error handling	<pre>If ["\$?" = "0"] then</pre>	<pre>try { Command } catch { errHandling commands }</pre>	<pre>begin statements rescue statements if error occurred else statements if no error end</pre>	<pre>try: statements raise customErrorObject except errorObject: statements except customErrorObject: statements finally: statements to clean up</pre>

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Arrays	<pre>bashArray = (val1, val2, val3) For l in 1 2 3 Do echo \${bashArray[\$i]} done</pre>	<pre>\$PSarray=@(1.3.5.7.9); for (\$i = 0; \$i -lt \$PSarray.Length; \$i++) { \$PSarray[\$i] } foreach (\$element in \$PSarray) { \$element }</pre>	<pre>rubyArray = ["val1", "val2", "val3"] print rubyArray[1] print rubyArray.index("val2")</pre>	<pre>pythonArray = [10, 20, 30, 40, 50] Print(pythonArray[1]) len(pythonArray)</pre>

Comparing Scripting Languages

	Bash	PowerShell	Ruby	Python
Encoding	Echo plainText base64	\$Text = 'Hello world' \$Bytes = [System.Text.Encoding]::Unicode.GetBytes(\$Text) \$EncodedText = [Convert]::ToBase64String(\$Bytes)	Require "base64" encString = Base64.encode64('Hello world!')	Import base64 encString = base64.encodestring('Hello world!')
Decoding	Echo encString base64 --decode	\$EncodedText = 'encodedString' \$DecodedText = [System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String(\$EncodedText))	plaintext = Base64.decode(encString)	plaintext = base64.decodestring(encString)

11.08

Episode **Data Structures, Part 1**
title:

Objective: **5.1 Explain the basic concepts of scripting and software development**

11.09

Episode **Data Structures, Part 2**
title:

Objective: **5.1 Explain the basic concepts of scripting and software development**

11.10

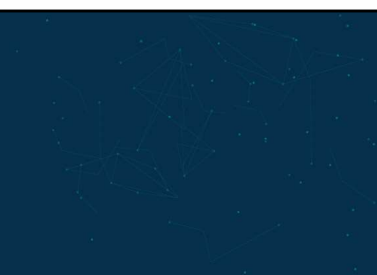
Episode **Libraries**
title:

Objective: **5.1 Explain the basic concepts of scripting and software development**

11.11

Episode **Classes**
title:

Objective: **5.1 Explain the basic concepts of scripting and software development**



11.12

Episode **Procedures and Functions**
title:

Objective: **5.1 Explain the basic concepts of scripting and software development**

11.13

Episode **Perl and JavaScript**
title:

Objective: **5.2 Given a scenario, analyze a script or code sample for use in a penetration test**

PERL

- **Perl – Flexible general purpose scripting language**
 - Actually, a family of 2 languages
 - Perl 5, just called Perl, and Perl 6, now called Raku
 - Originally developed as a UNIX scripting language for report processing
 - Perl and Raku now run on most operating systems

JAVASCRIPT

- **JavaScript – High level, just in time compiled language**
 - Popular in web applications
 - Flexible and powerful standalone language
 - Node.js – Popular open source JavaScript runtime environment

ADDITIONAL RESOURCES

- **Perl**

- <https://www.perl.org/>
- <https://github.com/dnmfarrell/Penetration-Testing-With-Perl>
- <https://github.com/lucthienphong1120/pentesting-scripts> DH1
- <https://highon.coffee/blog/penetration-testing-tools-cheat-sheet/>

- **JavaScript**

- <https://www.javascript.com/>
- <https://www.trustedsec.com/blog/javascript-essentials-for-beginning-pentesters> DH2
- <https://splicer.dev/writeups/javascript-for-pentesters-intro/> DH3
- <https://www.stationx.net/javascript-for-hacking/>

Slide 87

- DH1** Doesn't exist
Damon Hinton, 2025-08-07T22:24:30.469
- MS1 0** Replace with this: <https://github.com/dnmfarrell/Penetration-Testing-With-Perl>
Michael Solomon, 2025-08-12T20:56:57.718
- DH2** Doesn't exist anymore
Damon Hinton, 2025-08-07T22:31:24.716
- MS2 0** Replace with this: <https://www.trustedsec.com/blog/javascript-essentials-for-beginning-pentesters>
Michael Solomon, 2025-08-12T20:58:14.780
- DH3** Doesn't exist
Damon Hinton, 2025-08-07T22:32:40.180
- MS3 0** Replace with this: <https://www.stationx.net/javascript-for-hacking/>
Michael Solomon, 2025-08-12T20:59:17.780

PERL

```
#!/usr/bin/perl
use IO::Socket;
$| = 1;
print "IP : "; chop ($ip = <stdin>);
print "Start: "; chop ($startPort = <stdin>); print "End: "; chop ($endPort = <stdin>);

foreach ($port = $startPort ; $port <= $endPort ; $port++) {
    $socket = IO::Socket::INET->new(PeerAddr => $ip, PeerPort => $port, Proto => 'tcp',
    Timeout => 1);
    if( $socket ) {
        print "\r = Port $port is open.\n" ;
    }
    else { #Port is closed, silently move on }
}
exit (0);
```

JAVASCRIPT

```
var portscanner = require('portscanner');

// 127.0.0.1 is the default hostname
portscanner.findAPortNotInUse([3000, 3010],
  '127.0.0.1').then(port => {
  console.log(`Port ${port} is available!`);
});
```

11.14

Episode **Breach and Attack Simulation**
title:

Objective: **4.10.4**

CALDERA FRAMEWORK

- Developed by MITRE
- Automates post-exploitation
- Simulates MITRE tactics
- Creates custom procedures
- Popular in red teaming

INFECTION MONKEY

- Open-source tool
- Mimics attacker behavior
- Lateral movement, credential reuse
- Reports vulnerabilities
- Test segmentation controls

ATOMIC RED TEAM

- Scriptable attack technique
- Validates defense
- Runs tests in PowerShell, Bash
- Lightweight, reproducible tests